# Spring MVC Security - Basic Configuration - Overview and Demo

In this lesson, we will explore the fundamentals of basic security configuration by working through a practical example. Specifically, we'll be focusing on three users—John, Mary, and Susan—and setting up their respective roles as employee, manager, and admin.

| User ID | Password | Roles |
|---|---|---|
| john | test123 | EMPLOYEE |
| mary | test123 | EMPLOYEE, MANAGER |
| susan | test123 | EMPLOYEE, MANAGER, ADMIN |

You have the freedom to assign any names you desire to these roles; we've selected employee, manager, and admin for illustration purposes.

## Development Process Overview

1. **Creating a Spring Security Configuration Class:** This will allow us to define our security settings, including users' passwords and roles.
2. **Adding Users, Passwords, and Roles:** Here, we'll define our users in memory and associate them with specific roles.

## Step 1: Creating a Spring Security Configuration Class

The configuration class in Spring Security helps in defining the security attributes. We'll start by giving the class a configuration annotation. Then, within this class, we'll structure our security configurations, which will be elaborated in this lecture.

File: DemoSecurityConfig.java

```
import org.springframework.context.annotation.Configuration;

@Configuration
public class DemoSecurityConfig {

  // add our security configurations here …

}
```

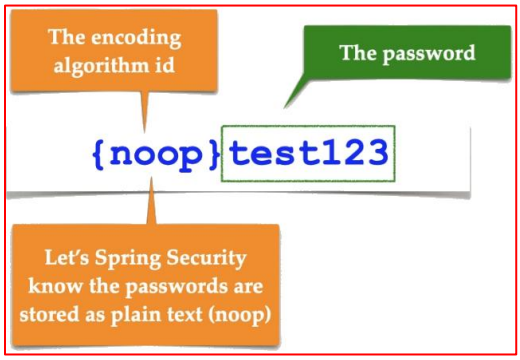
Spring Security stores passwords in a particular format. You'll encounter two types of encodings:




| ID | Description |
|---|---|
| noop | Plain text passwords |
| bcrypt | BCrypt password hashing |
| ... | ... |

- **Noop (No Operation):** This is simply plaintext, without any encryption or hashing.
- **Bcrypt:** A one-way hashing algorithm that encrypts the password.

For instance, a password like "test123" can be stored as "noop," indicating that it's in plaintext.




We'll begin with this method (noop), but we will also explore the Bcrypt method later in this series.

## Step 2: Adding Users, Passwords, and Roles

Now, let's define our users in memory. We'll use the `InMemoryUserDetailsManager` class and create our users, John, Mary, and Susan, with specific attributes, such as usernames, passwords, and roles.

Here's a code snippet to illustrate this process:

```
User john =
User.builder().username("john").password("test123").roles("employee").build();
// Repeat this process for Mary and Susan, including specific roles.
```

File: DemoSecurityConfig.java

```java
@Configuration
public class DemoSecurityConfig {

    @Bean
    public InMemoryUserDetailsManager userDetailsManager() {

        UserDetails john = User.builder()
                .username("john")
                .password("{noop}test123")
                .roles("EMPLOYEE")
                .build();

        UserDetails mary = User.builder()
                .username("mary")
                .password("{noop}test123")
                .roles("EMPLOYEE", "MANAGER")
                .build();

        UserDetails susan = User.builder()
                .username("susan")
                .password("{noop}test123")
                .roles("EMPLOYEE", "MANAGER", "ADMIN")
                .build();

        return new InMemoryUserDetailsManager(john, mary, susan);
    }
```

| User ID | Password | Roles |
|---|---|---|
| john | test123 | EMPLOYEE |
| mary | test123 | EMPLOYEE, MANAGER |
| susan | test123 | EMPLOYEE, MANAGER, ADMIN |

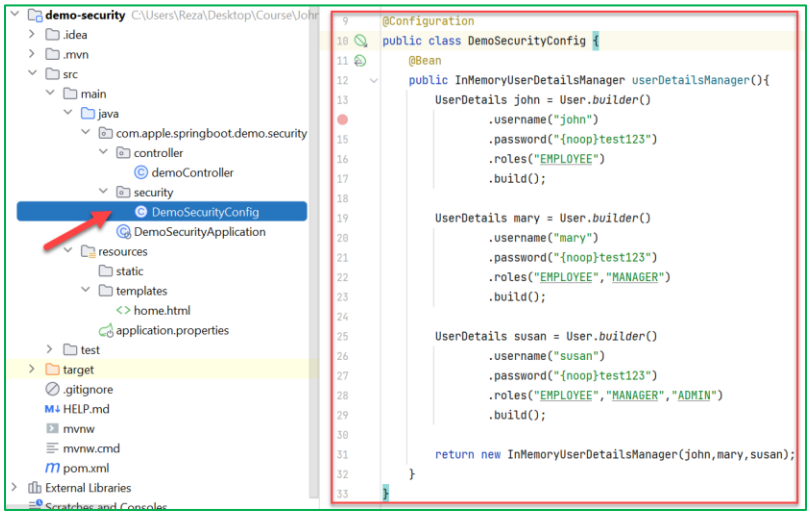
We'll return a new in-memory user details manager and pass in our three users. As we progress, we can include database support and cover techniques for storing passwords in plaintext and with Bcrypt encryption.

## Demonstrating the Process - Coding

Now, we'll actually write some code and implement the above steps. Here's a guide:

- Create a new package for security classes named "`security`."
- Inside this package, create a class named `DemoSecurityConfig`, and annotate it with the `@Configuration` annotation.
- Work through step two by adding the user's passwords and roles, utilizing the `InMemoryAuthentication`. You'll create user details for John, Mary, and Susan, then return an instance of `InMemoryUserDetailsManager` with the three user details.

Remember, this configuration overrides the default user passwords from the application properties, making it tailored to our specific needs.



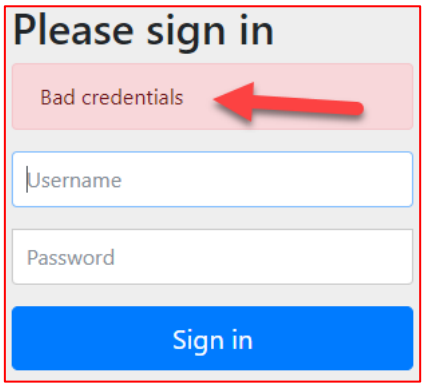
## Testing the Application

Finally, we'll test our application by attempting to log in with different user credentials. The Spring Security framework provides a default login page, and you'll notice immediate success or failure based on the information entered.

For instance:

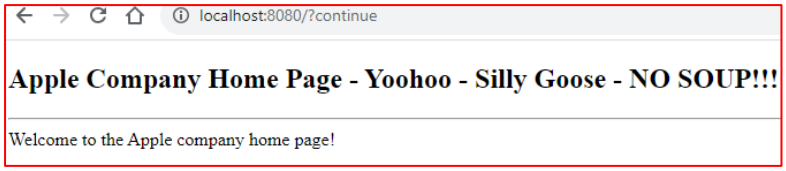
- Entering an incorrect user: "bad user" and password: "bad password" will result in a failure.
- Entering the correct username: "john" and password: "test123" will grant access.










## Summary

This lesson lays the groundwork for more advanced features, like showing or hiding information based on user roles. We've only scratched the surface; there's a lot more to explore.